

Домашняя работа 2

1. Установите ClickHouse.
2. Загрузите тестовый датасет и выполните выборку из таблицы.
3. Отправьте скриншоты работающего инстанса ClickHouse, созданной виртуальной машины (если выполняете работу в ЯО) и результата запроса: `select count() from trips where payment_type = 1`.
4. Проведите тестирование производительности и сохраните результаты.
5. Изучите конфигурационные файлы базы данных.
6. Настройте систему с учётом характеристик вашей ОС, оптимизируйте параметры и выполните повторное тестирование.
7. Подготовьте отчёт о приросте/изменении производительности системы на основе проведённых настроек. Укажите явно, какие параметры были изменены и почему.

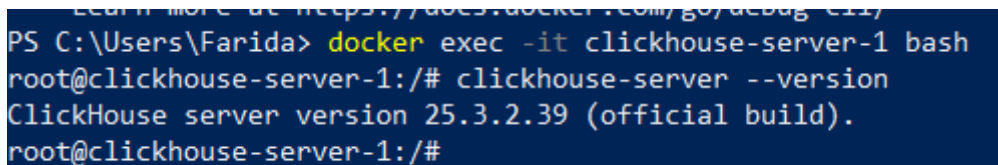
1. Установите clickhouse

Установку clickhouse я выполнила в докере.
Вот мой docker-compose.yml

```
version: '3.1'

services:
  clickhouse-server-1:
    image: clickhouse/clickhouse-server:latest
    container_name: clickhouse-server-1
    hostname: clickhouse-server-1
    ports:
      - "8123:8123" # HTTP порт
      - "9000:9000" # TCP порт
    volumes:
      - clickhouse_data-1:/var/lib/clickhouse # Хранение данных
      - clickhouse_config-1:/etc/clickhouse-server # Конфигурация
      (опционально)
      - /c/users/farida/downloads:/var/lib/clickhouse/files1 # Монтирование
        локальной директории в контейнер
    environment:
      CLICKHOUSE_USER: "click1" # Имя пользователя
      CLICKHOUSE_PASSWORD: "click1" # Пароль (если требуется)

volumes:
  clickhouse_data-1:
  clickhouse_config-1:
```



```
PS C:\Users\Farida> docker exec -it clickhouse-server-1 bash
root@clickhouse-server-1:/# clickhouse-server --version
ClickHouse server version 25.3.2.39 (official build).
root@clickhouse-server-1:/#
```

2. Загрузите тестовый датасет и выполните выборку из таблицы.

Сначала скачала тестовый дата-сет на локальный компьютер, подключила видимость директории windows, перенесла скачанный файл в докер-контейнер, потом производила загрузку через dbeaver.
Создала таблицу

```
CREATE TABLE uk.uk_price_paid_1
(
  `pk` String,
  `price` UInt32,
  `date` Date,
  `postcode1` LowCardinality(String),
  `postcode2` LowCardinality(String),
  `is_new` UInt8,
```

```

        `duration` LowCardinality(String),
        `addr1` String,
        `addr2` String,
        `street` LowCardinality(String),
        `locality` LowCardinality(String),
        `town` LowCardinality(String),
        `district` LowCardinality(String),
        `county` LowCardinality(String),
        `a` LowCardinality(String),
        `b` LowCardinality(String)
    )
ENGINE = MergeTree
ORDER BY (postcode1,
postcode2,
addr1,
addr2)
SETTINGS index_granularity = 8192;

Загрузила данные:
INSERT INTO uk.uk_price_paid_1
SELECT
    pk,
    price,
    toDate(substring(date, 1, 10)) AS date,
    postcode1,
    postcode2,
    if(is_new = 'Y', 1, 0) AS is_new,
    duration,
    addr1,
    addr2,
    street,
    locality,
    town,
    district,
    county,
    a, b
FROM file(
    'pp-complete.csv',
    'CSV',
    'pk String,
price UInt32,
date String,
postcode1 String,
postcode2 String,
is_new String,
duration String,
addr1 String,
addr2 String,
street String,
locality String,
town String,
district String,
county String, a String, b String');

```

Сделала выборку

```
clickhouse-server-1 :) select count(1) from uk.uk_price_paid_1 ;

SELECT count(1)
FROM uk.uk_price_paid_1

Query id: fef17e8d-d0d4-4fcb-8ab2-ec75f9deb9d5

1. count(1)
   39601751 -- 39.60 million

1 row in set. Elapsed: 0.004 sec.

clickhouse-server-1 :) █
```

3. Отправьте скриншоты работающего инстанса ClickHouse, созданной виртуальной машины (если выполняете работу в ЯО) и результата запроса: `select count() from trips where payment_type = 1`.

В bash сервера clickhouse выполнила замер производительности запроса с помощью benchmark:
`clickhouse-benchmark --query "select * from uk.uk_price_paid_1 upp where price<400000" -i 3`

```
root@clickhouse-server-1:/# clickhouse-benchmark --query "select * from uk.uk_price_paid_1 upp where price<400000" -i 3
Loaded 1 queries.

Queries executed: 1.

localhost:9000, queries: 1, QPS: 0.095, RPS: 3779313.099, MiB/s: 329.903, result RPS: 3253415.350, result MiB/s: 273.126.

0%      10.473 sec.
10%     10.473 sec.
20%     10.473 sec.
30%     10.473 sec.
40%     10.473 sec.
50%     10.473 sec.
60%     10.473 sec.
70%     10.473 sec.
80%     10.473 sec.
90%     10.473 sec.
95%     10.473 sec.
99%     10.473 sec.
99.9%   10.473 sec.
99.99%  10.473 sec.

Queries executed: 3.

localhost:9000, queries: 3, QPS: 0.105, RPS: 4141094.482, MiB/s: 361.483, result RPS: 3564854.247, result MiB/s: 299.272.

0%      9.036 sec.
10%     9.036 sec.
20%     9.036 sec.
30%     9.068 sec.
40%     9.068 sec.
50%     9.068 sec.
60%     9.068 sec.
70%     9.068 sec.
80%     10.473 sec.
90%     10.473 sec.
95%     10.473 sec.
99%     10.473 sec.
99.9%   10.473 sec.
99.99%  10.473 sec.
```

4. Проведите тестирование производительности и сохраните результаты.

Если выполнить тот же запрос на клиенте, то

Showed 1000 out of 34091101 rows.

34091101 rows in set. Elapsed: 7.887 sec. Processed 39.60 million rows, 3.62 GB (5.02 million rows/s., 459.58 MB/s.)
Peak memory usage: 53.89 MiB.

Как видим, запрос выполняется долго и просматриваются все строки, поскольку поле для отбора не входит в order by.

Если делать поиск по первому полю из order by, то запрос выполняется почти мгновенно и анализирует всего 16к строк

```
select count() from uk.uk_price_paid_1 upp where postcode1='CA2 7SD'
```

```
clickhouse-server-1 :) select count() from uk.uk_price_paid_1 upp where postcode1='CA2 7SD'

SELECT count()
FROM uk.uk_price_paid_1 AS upp
WHERE postcode1 = 'CA2 7SD'

Query id: bfa0b824-f0fb-4686-8895-9f92cfce14b1

1. count()
   115

1 row in set. Elapsed: 0.016 sec. Processed 16.38 thousand rows, 32.77 KB (1.03 million rows/s., 2.06 MB/s.)
Peak memory usage: 65.84 KiB.
```

Если делать поиск только по второму полю из order by, то анализируются все строки, но запрос все равно быстрый, поскольку мы считаем количество, которое хранится отдельно, и нам не надо залезать в сами данные

```
select count() from uk.uk_price_paid_1 upp where postcode2='T'
```

```
clickhouse-server-1 :) select count() from uk.uk_price_paid_1 upp where postcode2='T'

SELECT count()
FROM uk.uk_price_paid_1 AS upp
WHERE postcode2 = 'T'

Query id: 906f05f4-8bf0-45a9-9275-e53dafbdb632

1. count()
   11534846 -- 11.53 million

1 row in set. Elapsed: 0.040 sec. Processed 39.60 million rows, 39.60 MB (979.73 million rows/s., 979.73 MB/s.)
Peak memory usage: 104.98 KiB.
```

5. Изучите конфигурационные файлы базы данных.

Конфигурационные файлы лежат тут /etc/clickhouse-server/config.xml (настройки профилей пользователей тут /etc/clickhouse-server/users.xml)

6. Настройте систему с учётом характеристик вашей ОС, оптимизируйте параметры и выполните повторное тестирование.

Чтобы изменить конфигурацию, надо создать новый файл с новыми настройками тут /etc/clickhouse-server/config.d/config.xml (настройки профилей пользователей тут /etc/clickhouse-server/users.d/users.xml) Как нам говорили на следующей лекции, менять настройки придется только в каких-то экзотических случаях, поскольку настройки по умолчанию они для большинства случаев уже оптимальны. Поэтому задача заключалась в замедлении производительности.

Я меняла настройку max_server_memory_usage на 2000M, перезапустила сервер, убедилась, что настройка применилась:

```
clickhouse-server-1 :) select * from system.server_settings where name = 'max_server_memory_usage'

SELECT *
FROM system.server_settings
WHERE name = 'max_server_memory_usage'

Query id: 016c1d4c-dc4c-4100-8f03-d90a1ebe7dea

Row 1:
-----
name:          max_server_memory_usage
value:         2000000000
default:       0
changed:       1
description:    The maximum amount of memory the server is allowed to use, expressed in bytes.

:::note
The maximum memory consumption of the server is further restricted by setting 'max_server_memory_usage_to_ram_ratio'.
:::

As a special case, a value of '0' (default) means the server may consume all available memory (excluding further restrictions imposed by 'max_server_memory_usage_to_ram_ratio').

type:          UInt64
changeable_without_restart: Yes
is_obsolete:    0

1 row in set. Elapsed: 0.006 sec.
```

8. Подготовьте отчёт о приросте/изменении производительности системы на основе проведённых настроек. Укажите явно, какие параметры были изменены и почему.

Выполним запрос, который выполняли ранее max_server_memory_usage = 2000M

```
34091101 rows in set. Elapsed: 8.884 sec. Processed 39.60 million rows, 3.62 GB (4.46 million rows/s., 408.03 MB/s.)
Peak memory usage: 65.71 MiB.
```

Уменьшим max_server_memory_usage = 400M

```
SELECT *
FROM system.server_settings
WHERE name LIKE 'max_server_memory_usage'

Query id: c057dadd-533a-455f-b292-eee4da61f709

Row 1:
-----
name:          max_server_memory_usage
value:         400000000
default:       0
changed:       1
description:    The maximum amount of memory the server is allowed to use, expressed in bytes.

:::note
The maximum memory consumption of the server is further restricted by setting 'max_server_memory_usage_to_ram_ratio'.
:::

As a special case, a value of '0' (default) means the server may consume all available memory (excluding further restrictions imposed by 'max_server_memory_usage_to_ram_ratio').

type:          UInt64
changeable_without_restart: Yes
is_obsolete:    0
```

Попыталась выполнить запрос, который выполняла ранее select * from uk.uk_price_paid_1 upp where price<400000, получила MEMORY_LIMIT_EXCEEDED

```
clickhouse-server-1 :) select * from uk.uk_price_paid_1 upp where price<400000

SELECT *
FROM uk.uk_price_paid_1 AS upp
WHERE price < 400000

Query id: d1feb2b4-b7ee-4dd6-b224-86612d2fd19c

Elapsed: 0.026 sec.

Received exception from server (version 25.3.2):
Code: 241. DB::Exception: Received from localhost:9000. DB::Exception: (total) memory limit exceeded: would use 317.79 MiB (attempt to allocate chunk of 4.07 MiB bytes), current RSS: 512.26 MiB, maximum: 381.47 MiB. OvercommitTracker decision: Query was selected to stop by OvercommitTracker: While reading or decompressing /var/lib/clickhouse/store/37e/37e414b2-f2c0-4796-bf52-48b0b82ac06c/all_21_57_2/street.dict.bin (position: 78215, typename: DB::AsynchronousReadBufferFileWithDescriptorsCache, compressed data header: 20ea4f526ada9fd31af44b3c7cf79c20827731010009d20100): (while reading column street): (while reading from part /var/lib/clickhouse/store/37e/37e414b2-f2c0-4796-bf52-48b0b82ac06c/all_21_57_2/ in table uk.uk_price_paid_1 (37e414b2-f2c0-4796-bf52-48b0b82ac06c) located on disk default of type local, from mark 0 with max_rows_to_read = 8192): While executing MergeTreeSelect(pool: ReadPool, algorithm: Thread). (MEMORY_LIMIT_EXCEEDED)
```

Увеличила max_server_memory_usage = 1000M. Запрос начал выполняться, был выполнен частично, но упал из-за нехватки памяти MEMORY_LIMIT_EXCEEDED, но уже на другом этапе:

```
Showing 1000 out of 2633266 rows.

2633266 rows in set. Elapsed: 0.933 sec. Processed 2.88 million rows, 264.25 MB (3.08 million rows/s., 283.36 MB/s.)
Peak memory usage: 45.53 MiB.

Received exception from server (version 25.3.2):
Code: 241. DB::Exception: Received from localhost:9000. DB::Exception: (total) memory limit exceeded: would use 262.99 MiB (attempt to allocate chunk of 4.01 MiB bytes), current RSS: 950.96 MiB, maximum: 953.67 MiB. OvercommitTracker decision: Query was selected to stop by OvercommitTracker: While executing MergeTreeSelect(pool: ReadPool, algorithm: Thread). (MEMORY_LIMIT_EXCEEDED)
```

Запрос успешно выполнен при значении max_server_memory_usage = 1200M

```
country |> summarise(
  price = sum(price)
)
Showed 1000 out of 34091101 rows.

34091101 rows in set. Elapsed: 7.943 sec. Processed 39.60 million rows, 3.62 GB (4.99 million rows/s., 456.33 MB/s.)
Peak memory usage: 53.00 MiB.
```

Вернем исходные настройки (использовать всю имеющуюся память) max_server_memory_usage = 0.
Почему-то запрос выполнен медленнее:

```
pk |> summarise(
  price = sum(price)
)
Showed 1000 out of 34091101 rows.

34091101 rows in set. Elapsed: 8.170 sec. Processed 39.60 million rows, 3.62 GB (4.85 million rows/s., 443.70 MB/s.)
Peak memory usage: 51.38 MiB.
```